AE 370 Group Project 1

Aathreya Doddi, Adam Essa, Burak Kirali, Abel Yohannes

October 2026

1 Premise

Orbital decay is a key part of understanding how an orbit will progress over time, especially for satellites that focus on long term missions. In particular, satellites in LEO ($< 2000\,km$) plan for missions that can last years or even decades, requiring energy generation in terms of solar panels that affect drag. While models like the two body problem attempt to solve the trajectories of objects with closed form analytical solutions, this project attempts to solve for orbital decay in terms of gravity and atmospheric drag (nonexistent in the two body problem), while accounting for solar panel orientation. This was chosen to better understand the situations in which engineering design decisions affect mission success. Orbital decay itself can shorten a satellite's lifetime and can prevent it from fulfilling the mission. Understanding the relationship helps engineers choose their initial conditions and design configurations to achieve a set mission lifetime. Solar panels are a popular choice, yet they directly affect orbital decay and how long a mission can operate for. Investigating how initial conditions and design choices affect mission lifetime is our premise. Specifically, we want to answer: how does an initial orbital altitude and solar panel orientation affect orbital decay in LEO subject to gravity and atmospheric drag. The goal is to determine the time it takes for the satellite to reach 120 km and the velocity at that altitude. This requires solving the IVP:

$$\frac{dx}{dt} = v_x, \qquad \frac{dy}{dt} = v_y$$

$$\frac{dv_x}{dt} = -\frac{\mu x}{r^3} - \frac{C_D A(\theta)\rho(h)}{2m}|v|v_x$$

$$\frac{dv_y}{dt} = -\frac{\mu y}{r^3} - \frac{C_D A(\theta)\rho(h)}{2m}|v|v_y$$

where

$$r = \sqrt{x^2 + y^2}, \qquad h = r - R_E,$$

$$|v| = \sqrt{{v_x}^2 + {v_y}^2}, \qquad A(\theta) = \begin{cases} A_s + A_p sin^2(\theta), & 0 \le \theta < \pi \\ A_s, & \pi \le \theta < 2\pi \end{cases}$$

Initial Conditions:

$$\begin{aligned} x(0) &= R_E + h_0 \\ y(0) &= 0 \\ v_x(0) &= 0 \\ v_y(0) &= \sqrt{\frac{\mu}{R_E + h_0}} \end{aligned}$$

Parameters:

$\mu = GM_E$:	Gravitational parameter of Earth
R_E :	Radius of Earth
m :	Mass of the satellite
$A(\theta)$:	Total cross sectional area of the satellite
A_s :	Cross sectional area of the satellite body
A_p :	Maximum additional solar panel area
$\theta = atan2(y, x)\ (mod 2\pi)$:	Angular position of the satellite
C_D :	Coefficient of drag
$\rho(h)$:	Atmospheric density

Addressing our question requires studying this IVP because of the nature of the problem. Drag is a force that is dependent on velocity, so when creating the equations of motion it creates a situation where the acceleration of the body is related to the velocity. Also, the velocity terms in the drag equation result in a nonlinear IVP. Nonlinear equations, altitude dependent air density, and varying cross sectional area make a closed form analytical solution extremely difficult and impractical. This motivates us to turn towards a numerical method to solve this IVP

We assume that the satellite has a constant mass and drag coefficient. Also, Earth is spherical, we start at an initial circular orbit, we have 2D motion, and both the Earth and atmosphere are fixed relative to our coordinate system. 2D motion was chosen as we can model any orbit in its orbital plane. Air density is calculated with the US Standard Atmosphere, which is used for orbital decay and atmospheric drag, and it is stationary. Also, the cross sectional area equation is used to model the solar panel orientation. There will always be the satellite's own cross sectional area, but from 0 to 180 degrees of our orbit (sun side), the panels will operate in order to capture sunlight. This is modeled as a sine function as it has the desired change in area from 0 to 90 to 180 degrees. This is an assumed deployment strategy where the total cross sectional geometry will increase and decrease as the panels are deployed and retracted to gain more sunlight along the orbital path. Note that while the initial orbit is circular, the satellite is able to develop a non-circular trajectory as it decays

2 Method

2.1 Stiffness Analysis

Before selecting a numerical integration method, we first investigate whether the system governing orbital decay exhibits stiff behavior. Stiffness is important because it can impose restrictive timestep requirements on explicit numerical methods, increasing computational cost. Analyzing the stiffness of our system will help us select an appropriate numerical method based on stability, accuracy, and efficiency.

We begin by expressing our initial value problem as a system of first-order ordinary differential equations:

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(\mathbf{u}) \tag{1}$$

where the state vector is

$$\mathbf{u} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix}. \tag{2}$$

For convenience, we define

$$q = \frac{C_D A(\theta)\rho(h)}{2m}, \qquad s = \sqrt{v_x^2 + v_y^2}. \tag{3}$$

Using these definitions, the governing equations become

$$\mathbf{f}(\mathbf{u}) = \begin{bmatrix} v_x \\ v_y \\ -\dfrac{\mu x}{r^3} - qsv_x \\ -\dfrac{\mu y}{r^3} - qsv_y \end{bmatrix}. \tag{4}$$

To investigate the local behavior of the system, we construct the Jacobian matrix, defined as

$$J(\mathbf{u}) = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}. \tag{5}$$

Since our system contains four state variables, the Jacobian is a 4×4 matrix:

$$J = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ B_{11} & B_{12} & C_{11} & C_{12} \\ B_{21} & B_{22} & C_{21} & C_{22} \end{bmatrix}. \tag{6}$$

The first two rows follow directly from $\dot{x} = v_x$ and $\dot{y} = v_y$.

The remaining entries are obtained by differentiating the gravitational and drag acceleration terms with respect to position and velocity.

For the gravitational acceleration,

$$a_{x,g} = -\frac{\mu x}{r^3}, \qquad a_{y,g} = -\frac{\mu y}{r^3}, \tag{7}$$

where $r = \sqrt{x^2 + y^2}$.
Applying the product and chain rules gives

$$\frac{\partial a_{x,g}}{\partial x} = -\frac{\mu}{r^3} + \frac{3\mu x^2}{r^5}, \tag{8}$$

$$\frac{\partial a_{x,g}}{\partial y} = \frac{3\mu xy}{r^5}, \tag{9}$$

$$\frac{\partial a_{y,g}}{\partial x} = \frac{3\mu xy}{r^5}, \tag{10}$$

$$\frac{\partial a_{y,g}}{\partial y} = -\frac{\mu}{r^3} + \frac{3\mu y^2}{r^5}. \tag{11}$$

For atmospheric drag, we have

$$a_{x,d} = -qsv_x, \qquad a_{y,d} = -qsv_y. \tag{12}$$

Since $s = \sqrt{v_x^2 + v_y^2}$, its derivatives are

$$\frac{\partial s}{\partial v_x} = \frac{v_x}{s}, \qquad \frac{\partial s}{\partial v_y} = \frac{v_y}{s}. \tag{13}$$

Differentiating the drag acceleration with respect to velocity gives

$$C_{11} = -q\left(s + \frac{v_x^2}{s}\right), \tag{14}$$

$$C_{12} = -q\frac{v_x v_y}{s}, \tag{15}$$

$$C_{21} = -q\frac{v_x v_y}{s}, \tag{16}$$

$$C_{22} = -q\left(s + \frac{v_y^2}{s}\right). \tag{17}$$

The drag coefficient q also depends on position through the atmospheric density and solar panel area. Therefore, its position derivatives must be included.

$$\rho(h) = \rho_0 e^{-(h-h_{ref})/H}, \qquad h = r - R_E. \tag{18}$$

In our implementation, atmospheric density is calculated using logarithmic interpolation between tabulated values. Therefore, the exponential density model is applied piecewise, with ρ_0, h_{ref}, and H varying between altitude intervals.

Differentiating the density function gives

$$\frac{\partial \rho}{\partial x} = -\frac{\rho x}{Hr}, \qquad \frac{\partial \rho}{\partial y} = -\frac{\rho y}{Hr}. \tag{19}$$

The orbital angle is defined as

$$\theta = \operatorname{atan2}(y, x) \pmod{2\pi}. \tag{20}$$

Its position derivatives are

$$\frac{\partial \theta}{\partial x} = -\frac{y}{r^2}, \qquad \frac{\partial \theta}{\partial y} = \frac{x}{r^2}. \tag{21}$$

From our solar panel area model,

$$A_\theta = \frac{dA}{d\theta} = \begin{cases} A_p \sin(2\theta), & 0 \le \theta < \pi, \\ 0, & \pi \le \theta < 2\pi. \end{cases} \tag{22}$$

Using the product and chain rules, we obtain

$$q_x = \frac{\partial q}{\partial x} = -\frac{qx}{Hr} - \frac{C_D \rho(h) A_\theta y}{2mr^2}, \tag{23}$$

$$q_y = \frac{\partial q}{\partial y} = -\frac{qy}{Hr} + \frac{C_D \rho(h) A_\theta x}{2mr^2}. \tag{24}$$

Combining the gravitational and drag contributions, the position derivative entries become

$$B_{11} = -\frac{\mu}{r^3} + \frac{3\mu x^2}{r^5} - s v_x q_x, \tag{25}$$

$$B_{12} = \frac{3\mu xy}{r^5} - s v_x q_y, \tag{26}$$

$$B_{21} = \frac{3\mu xy}{r^5} - s v_y q_x, \tag{27}$$

$$B_{22} = -\frac{\mu}{r^3} + \frac{3\mu y^2}{r^5} - s v_y q_y. \tag{28}$$

These expressions complete the Jacobian matrix of our orbital decay system.

To investigate potential stiffness, we examine the eigenvalues of the Jacobian, which satisfy

$$\det(J - \lambda I) = 0. \tag{29}$$

The eigenvalues describe the local linearized behavior of the system. Eigenvalues associated with rapidly decaying modes can impose restrictive stability requirements on explicit integration methods.

We will evaluate the Jacobian at representative states along the satellite trajectory and examine the resulting eigenvalues. Since the Jacobian varies throughout the orbit, evaluating it at multiple states is necessary.

We will then compare timestep restrictions imposed by numerical stability with those required to achieve sufficient accuracy.

The results of this analysis will guide our choice of numerical integration method.

2.2 Eigenvalue Evaluation

To assess the stiffness of the orbital decay system, we examine the eigenvalues of the Jacobian matrix. These eigenvalues characterize the local behavior of the system and provide insight into the stability requirements of numerical integration methods.

We first evaluate the Jacobian at the initial conditions defined in the premise:

$$\begin{aligned} x(0) &= R_E + h_0, \\ y(0) &= 0, \\ v_x(0) &= 0, \\ v_y(0) &= \sqrt{\frac{\mu}{R_E + h_0}}. \end{aligned}$$

Defining the initial orbital radius, velocity, and angular frequency as

$$\begin{aligned} r_0 &= R_E + h_0, \\ v_0 &= \sqrt{\frac{\mu}{r_0}}, \\ \omega_0 &= \sqrt{\frac{\mu}{r_0^3}}, \end{aligned}$$

we can simplify the Jacobian at the initial state. Since the initial angular position is $\theta = 0$, the projected area is $A(0) = A_s$. Therefore,

$$\begin{aligned} q_0 &= \frac{C_D A_s \rho_0}{2m} e^{-(h_0 - h_{ref})/H}, \\ \gamma_0 &= q_0 v_0. \end{aligned}$$

Substituting the initial conditions into the previously derived Jacobian gives

$$J_0 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 2\omega_0^2 & 0 & -\gamma_0 & 0 \\ \dfrac{q_0 v_0^2}{H} & -\omega_0^2 & 0 & -2\gamma_0 \end{bmatrix}.$$

The eigenvalues are obtained by solving the characteristic equation

$$\det(J_0 - \lambda I) = 0.$$

Expanding the determinant and factoring yields

$$(\lambda^2 + \gamma_0\lambda - 2\omega_0^2)(\lambda^2 + 2\gamma_0\lambda + \omega_0^2) = 0.$$

Solving these quadratic equations gives the four eigenvalues:

$$\lambda_{1,2} = \frac{-\gamma_0 \pm \sqrt{\gamma_0^2 + 8\omega_0^2}}{2},$$

$$\lambda_{3,4} = -\gamma_0 \pm \sqrt{\gamma_0^2 - \omega_0^2}.$$

The eigenvalues depend on the initial orbital altitude and the atmospheric drag parameters through ω_0 and γ_0.

To examine the limiting behavior, we consider the case in which atmospheric drag is neglected. Setting $C_D = 0$ gives $\gamma_0 = 0$, reducing the eigenvalues to

$$\lambda_{1,2} = \pm\sqrt{2}\omega_0,$$
$$\lambda_{3,4} = \pm i\omega_0.$$

These correspond to the eigenvalues of the instantaneous Jacobian for the drag-free two-body system. The positive real eigenvalue does not necessarily indicate an unstable orbit, as the Jacobian is evaluated along a moving trajectory rather than at an equilibrium point.

Since atmospheric density and projected solar panel area vary throughout the orbit, the Jacobian and its eigenvalues also change with the satellite's position and velocity. Therefore, the initial-state eigenvalues alone are insufficient to classify the system as stiff or nonstiff.

A numerical stiffness assessment will consequently require evaluating the Jacobian at representative states along the trajectory. The resulting eigenvalues will be used to investigate the stability restrictions of explicit integration methods and compare them with the timestep requirements for numerical accuracy. This comparison will provide the basis for selecting an appropriate numerical integration method.

2.3 Numerical Stiffness Assessment

To determine whether the orbital decay system exhibits stiff behavior, we compare the timestep restrictions imposed by numerical stability with those required for accuracy.

We begin by considering the linear test equation

$$\frac{du}{dt} = \lambda u,$$

where λ represents an eigenvalue of the Jacobian. Applying a numerical integration method to this equation gives

$$u_{k+1} = R(\lambda \Delta t) u_k,$$

where R is the stability function and Δt is the timestep.

For modes with negative real eigenvalues, numerical stability requires

$$|R(\lambda \Delta t)| \leq 1.$$

For Forward Euler, the stability function is

$$R(z) = 1 + z.$$

Therefore, its stability condition becomes

$$|1 + \lambda \Delta t| \leq 1.$$

For the classical fourth-order Runge-Kutta method, the stability function is

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.$$

Its corresponding stability condition is

$$\left| 1 + \lambda \Delta t + \frac{(\lambda \Delta t)^2}{2} + \frac{(\lambda \Delta t)^3}{6} + \frac{(\lambda \Delta t)^4}{24} \right| \leq 1.$$

Using these conditions, we can estimate the timestep restrictions associated with the eigenvalues at different points along the orbit.

Since the Jacobian changes with altitude and orbital position, we evaluate its eigenvalues at representative states throughout the satellite's trajectory. Particular attention is given to lower altitudes, where atmospheric density and drag increase.

However, because the Jacobian is evaluated along a moving trajectory rather than at an equilibrium point, local eigenvalue conditions alone cannot establish the stability of the complete nonlinear system.

We therefore supplement the eigenvalue analysis with numerical timestep refinement studies. The accuracy of each method is evaluated using the predicted time required to reach an altitude of 180 km and the satellite's speed at 120 km.

For the predicted orbital lifetime, the relative error is defined as

$$\epsilon_T = \frac{|T_{\Delta t} - T_{\text{ref}}|}{|T_{\text{ref}}|},$$

where T_{ref} is a converged reference solution obtained using a sufficiently small timestep.

Similarly, the relative error in velocity at 120 km is

$$\epsilon_v = \frac{|v_{\Delta t} - v_{\text{ref}}|}{|v_{\text{ref}}|}.$$

By reducing the timestep and examining these errors, we can determine the timestep needed to obtain accurate results.

If the timestep required for numerical stability is significantly smaller than that required for accuracy, this would indicate potentially stiff behavior. Otherwise, an explicit method may be sufficient.

The combined stability and accuracy analysis will be used to select a numerical method that provides reliable orbital decay predictions at a reasonable computational cost.

2.4 Numerical Method Selection and Justification

Based on the stiffness analysis in the previous section,

RK4 has a higher order accuracy than Forward Euler, having a global truncation error of $O(\Delta t^4)$, compared to the latter's $O(\Delta t)$. As a result, halving the timestep reduces the global error by a factor of 16 for RK4, while Forward Euler only sees a reduction by a factor of 2. A faster reduction in numerical error is suitable for accurately predicting orbital decay over extended simulation periods. RK4's superior accuracy, however, comes at a higher computational cost, as each timestep evaluates four functions unlike Forward Euler, which only requires one evaluation. Although RK4 is more expensive than Forward Euler per timestep, its higher accuracy could require far fewer timesteps to achieve the same level of accuracy.

Considering these factors, we propose RK4 as the primary numerical integration method for our orbital decay simulation. Its final suitability and timestep requirements will be evaluated through numerical convergence and stability studies. The mathematical derivation of RK4 and its error and stability properties are presented in the following subsection.

2.5 Mathematical Derivation of RK4

We consider the classical fourth-order Runge–Kutta (RK4) method to solve the orbital decay initial value problem defined in Section 2.1.
Consider a general initial value problem of the form

$$\frac{du}{dt} = f(t, u), \tag{30}$$

with initial condition

$$u(t_0) = u_0. \tag{31}$$

Suppose the numerical solution u_k is known at time t_k. We wish to approximate the solution at

$$t_{k+1} = t_k + \Delta t. \tag{32}$$

For convenience, let $h = \Delta t$.

Integrating the governing equation from t_k to $t_k + h$ gives

$$\int_{t_k}^{t_k+h} \frac{du}{dt}\, dt = \int_{t_k}^{t_k+h} f(t, u(t))\, dt. \tag{33}$$

Therefore,

$$\boxed{u(t_k + h) = u(t_k) + \int_{t_k}^{t_k+h} f(t, u(t))\, dt}. \tag{34}$$

The integral cannot generally be evaluated directly because the solution $u(t)$ is unknown.

Numerical integration methods approximate this integral using evaluations of the derivative function f.

For example, Forward Euler approximates the integral using only the derivative at the beginning of the timestep:

$$u_{k+1} = u_k + hf(t_k, u_k). \tag{35}$$

The fourth-order Runge–Kutta method instead uses four derivative evaluations to obtain a higher-order approximation. For a general system $\mathbf{u}' = \mathbf{f}(\mathbf{u})$, an explicit four-stage Runge–Kutta method can be written as:

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{u}_k), \tag{36}$$

$$\mathbf{k}_2 = \mathbf{f}(\mathbf{u}_k + ha_{21}\mathbf{k}_1), \tag{37}$$

$$\mathbf{k}_3 = \mathbf{f}(\mathbf{u}_k + h(a_{31}\mathbf{k}_1 + a_{32}\mathbf{k}_2)), \tag{38}$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{u}_k + h(a_{41}\mathbf{k}_1 + a_{42}\mathbf{k}_2 + a_{43}\mathbf{k}_3)). \tag{39}$$

The solution is advanced using

$$\mathbf{u}_{k+1} = \mathbf{u}_k + h\sum_{i=1}^{4} b_i\mathbf{k}_i. \tag{40}$$

For classical RK4, the intermediate stages are chosen at the beginning, midpoint, midpoint, and end of the timestep. Specifically,

$$a_{21} = \frac{1}{2}, \qquad a_{32} = \frac{1}{2}, \qquad a_{43} = 1, \tag{41}$$

with all remaining $a_{ij} = 0$. The corresponding stage locations are

$$c_1 = 0, \quad c_2 = \frac{1}{2}, \quad c_3 = \frac{1}{2}, \quad c_4 = 1. \tag{42}$$

The stage weights are determined by requiring the numerical update to match the Taylor expansion of the exact solution through fourth order. The order conditions include

$$\sum_i b_i = 1, \tag{43}$$

$$\sum_i b_i c_i = \frac{1}{2}, \tag{44}$$

$$\sum_i b_i c_i^2 = \frac{1}{3}, \tag{45}$$

$$\sum_i b_i c_i^3 = \frac{1}{4}. \tag{46}$$

Substituting the stage locations gives

$$b_1 + b_2 + b_3 + b_4 = 1, \tag{47}$$

$$\frac{1}{2}(b_2 + b_3) + b_4 = \frac{1}{2}, \tag{48}$$

$$\frac{1}{4}(b_2 + b_3) + b_4 = \frac{1}{3}, \tag{49}$$

$$\frac{1}{8}(b_2 + b_3) + b_4 = \frac{1}{4}. \tag{50}$$

Subtracting the third equation from the second yields

$$b_2 + b_3 = \frac{2}{3}. \tag{51}$$

Substitution into the third and first equations then gives

$$b_4 = \frac{1}{6}, \qquad b_1 = \frac{1}{6}. \tag{52}$$

An additional fourth-order condition is

$$\sum_i b_i \sum_j a_{ij} c_j^2 = \frac{1}{12}. \tag{53}$$

For the chosen stage coefficients,

$$\frac{1}{8} b_3 + \frac{1}{4} b_4 = \frac{1}{12}, \tag{54}$$

which gives

$$b_3 = \frac{1}{3}, \qquad b_2 = \frac{1}{3}. \tag{55}$$

These coefficients also satisfy the remaining fourth-order Runge–Kutta conditions.

Therefore, the classical RK4 equations are

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{u}_k), \tag{56}$$

$$\mathbf{k}_2 = \mathbf{f}\left(\mathbf{u}_k + \frac{h}{2}\mathbf{k}_1\right), \tag{57}$$

$$\mathbf{k}_3 = \mathbf{f}\left(\mathbf{u}_k + \frac{h}{2}\mathbf{k}_2\right), \tag{58}$$

$$\mathbf{k}_4 = \mathbf{f}\left(\mathbf{u}_k + h\mathbf{k}_3\right). \tag{59}$$

The final update becomes

$$\boxed{\mathbf{u}_{k+1} = \mathbf{u}_k + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)}. \tag{60}$$

For our orbital decay problem, each stage evaluates the governing equations using the corresponding intermediate position and velocity. The orbital radius, atmospheric density, and projected area are therefore recomputed at every stage.

2.5.1 Error Properties

To determine the error order of RK4, we expand the exact solution about t_k using Taylor's theorem. For a sufficiently smooth solution $\mathbf{u}(t)$, the Taylor expansion is given by

$$\mathbf{u}(t_k + h) = \sum_{n=0}^{4} \frac{h^n}{n!} \mathbf{u}^{(n)}(t_k) + O(h^5). \tag{61}$$

Expanding the summation and using $\mathbf{u}^{(0)} = \mathbf{u}$ gives

$$\begin{aligned} \mathbf{u}(t_k + h) = \mathbf{u}(t_k) + h\mathbf{u}'(t_k) + \frac{h^2}{2!}\mathbf{u}''(t_k) \\ + \frac{h^3}{3!}\mathbf{u}'''(t_k) + \frac{h^4}{4!}\mathbf{u}^{(4)}(t_k) + O(h^5). \end{aligned} \tag{62}$$

The RK4 coefficients are chosen such that the numerical update matches the exact Taylor expansion through terms of order h^4.

Since classical RK4 satisfies the fourth-order conditions, its local truncation error is

$$\boxed{\tau_{\text{local}} = O(h^5)}. \tag{63}$$

For a sufficiently smooth solution on a fixed integration interval, the global error is

$$\boxed{e_{\text{global}} = O(h^4)}. \tag{64}$$

Consequently, halving the timestep is expected to reduce the global error by approximately a factor of $2^4 = 16$ in the asymptotic regime.

2.5.2 Stability Derivation

Applying RK4 to the linear test equation $u' = \lambda u$, with $z = \lambda h$, gives

$$k_1 = \lambda u_k, \tag{65}$$

$$k_2 = \lambda u_k \left(1 + \frac{z}{2}\right), \tag{66}$$

$$k_3 = \lambda u_k \left(1 + \frac{z}{2} + \frac{z^2}{4}\right), \tag{67}$$

$$k_4 = \lambda u_k \left(1 + z + \frac{z^2}{2} + \frac{z^3}{4}\right). \tag{68}$$

Substituting into the RK4 update and collecting powers of z yields

$$u_{k+1} = \left(1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}\right) u_k. \tag{69}$$

Thus, the stability function is

$$\boxed{R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}}. \tag{70}$$

For decaying modes, absolute stability requires $|R(\lambda h)| \leq 1$, consistent with the stability analysis in Section 2.3.

Each RK4 timestep requires four evaluations of the governing equations, giving a total cost of $4N$ evaluations for N steps. This additional per-step cost is balanced against its fourth-order accuracy and larger stability region relative to Forward Euler.

3 RK4 Implementation

We implement the classical fourth-order Runge–Kutta (RK4) method using the formulation described above. Our code executes RK4 inside a while loop. After each timestep, we check whether the satellite's altitude has fallen below the specified threshold. If the threshold has not been reached, the code continues performing RK4 steps until the satellite reaches the cutoff altitude.

3.0.1 Validation

Before trusting the results obtained from our algorithm, we need to investigate two things: whether the results depend on the choice of timestep and whether the numerical solution is accurate.

To test the accuracy of our implementation, we use two verification methods. First, if atmospheric drag is neglected, a satellite initialized in a circular orbit follows a mathematically known trajectory. We can therefore compare the results obtained from RK4 against the exact analytical solution.

Second, when atmospheric drag is included, the decrease in the satellite's total mechanical energy should equal the work done by atmospheric drag. Comparing these two energy changes allows us to verify the physical consistency of our numerical solution.

3.0.2 No Atmospheric Drag Case

To test the accuracy of RK4 for circular motion, we compute the position and velocity errors after one complete orbit at an initial altitude of 400 km for various timesteps.

As shown in the figure, larger timesteps, such as 120 s and 240 s, produce significant position errors, making them unsuitable for accurate long-term orbital simulations. However, as the timestep is reduced to 30 s and 15 s, the position error decreases substantially, reaching the sub-meter scale.

By comparing the reduction in position error to the reduction in timestep, we can estimate the observed order of convergence:

$$p = \frac{\ln(e_{\Delta t}/e_{\Delta t/2})}{\ln(2)} \tag{71}$$

where $e_{\Delta t}$ is the position error obtained using timestep Δt. From our results, the observed convergence order is approximately four, consistent with the theoretical fourth-order accuracy of RK4.

```
dt (s)   Position error (m)   Velocity error (m/s)
 240.0       1.35713117e+04       1.52645696e+01
 120.0       5.91551840e+02       6.67010484e-01
  60.0       2.89412509e+01       3.26823741e-02
  30.0       1.55589752e+00       1.75991070e-03
  15.0       8.93240065e-02       1.01115011e-04

========== TEST 2: RK4 ORDER ==========
Expected global order: p = 4

dt -> dt/2       Observed order
240.0 -> 120.0       4.51991
120.0 ->  60.0       4.35331
 60.0 ->  30.0       4.21731
 30.0 ->  15.0       4.12256
```

Figure 1: RK4 accuracy for a 400 km circular orbit.

```
========== TEST 3: ENERGY CONSERVATION ==========
dt (s)   Maximum relative energy deviation
 120.0   8.16499416e-05
  60.0   2.54056358e-06
  30.0   7.92927211e-08
  15.0   2.47678506e-09
```

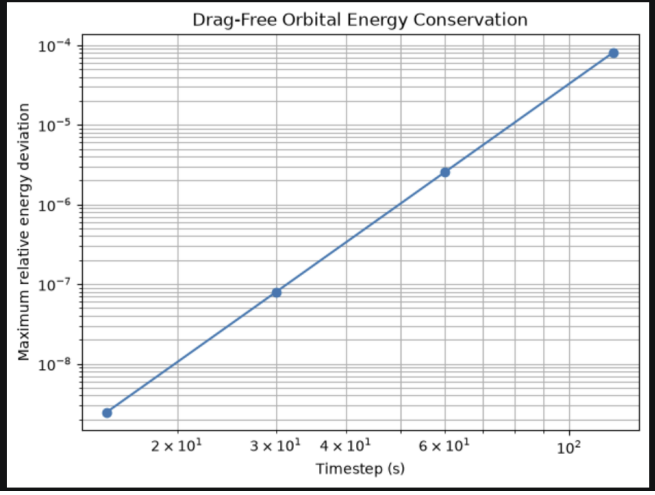


Figure 2: RK4 energy deviation in a drag free orbit.

We also conduct an energy conservation study for the drag-free orbit. Since gravity is a conservative force, the satellite's total mechanical energy should remain constant throughout the simulation.

The specific orbital mechanical energy is given by

$$E = \frac{v^2}{2} - \frac{\mu}{r} \tag{72}$$

where v is the satellite's velocity magnitude, r is its distance from Earth's center, and μ is Earth's gravitational parameter.

We simulate ten complete orbits and calculate the maximum relative deviation in mechanical energy. The results show that the numerical energy drift becomes extremely small for timesteps such as 15 s, providing additional evidence that our RK4 implementation accurately reproduces drag-free orbital motion.

3.1 Energy Loss Due to Atmospheric Drag

Having established that RK4 accurately reproduces circular orbital motion for sufficiently small timesteps, we perform another verification study with atmospheric drag included.

In this case, the satellite loses mechanical energy due to the work performed by atmospheric drag. According to the work-energy principle, the change in orbital mechanical energy should equal the total work done by the drag force.

To verify this, we simulate one orbital period starting from an initial circular orbit at an altitude of 200 km.

We first calculate the satellite's specific mechanical energy at the beginning and end of the simulation:

$$\Delta E_{\text{orbit}} = E_{\text{final}} - E_{\text{initial}} \tag{73}$$

We then calculate the specific energy removed by atmospheric drag by integrating the drag power over the same time interval.

The atmospheric drag force magnitude is

$$F_D = \frac{1}{2}\rho v^2 C_D A \tag{74}$$

Since drag acts opposite to the direction of motion, the rate of change of specific mechanical energy is

$$\frac{dE}{dt} = -\frac{C_D A \rho v^3}{2m} \tag{75}$$

Therefore, the total specific energy change due to drag is

$$\Delta E_{\text{drag}} = -\int_0^T \frac{C_D A(\theta(t))\rho(t)v(t)^3}{2m}\, dt \tag{76}$$

We numerically integrate this expression using the same RK4 stages as the orbital integration. We then compare the result to the change in mechanical energy calculated directly from the satellite's initial and final states.

The relative energy-balance error is calculated as

$$\epsilon_E = \frac{|\Delta E_{\text{orbit}} - \Delta E_{\text{drag}}|}{|\Delta E_{\text{drag}}|} \tag{77}$$

```
========== TEST 7: DRAG ENERGY BALANCE ==========
One orbit at 200 km

dt(s)   State energy loss (J/kg)   Drag integral (J/kg)   Rel. mismatch
120.0            -8435.49475991         -8112.51675592   3.98123066e-02
 60.0            -7231.17308417         -7220.74077505   1.44476993e-03
 30.0            -7161.55662647         -7161.20500535   4.91008325e-05
 15.0            -7155.55360267         -7155.54097098   1.76530204e-06
  7.5            -7154.91353667         -7154.91303982   6.94415996e-08

Observed energy-balance convergence order:
120 -> 60 s: p = 4.7843
60 -> 30 s: p = 4.8789
30 -> 15 s: p = 4.7978
15 -> 7.5 s: p = 4.6680
```

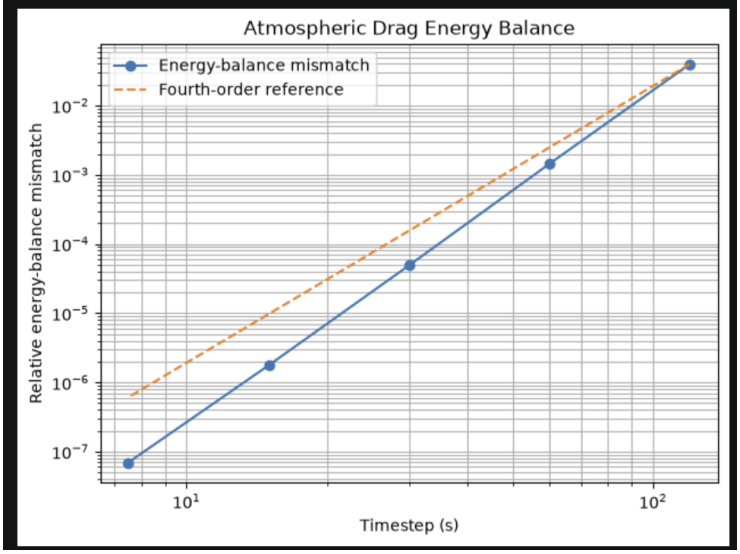


Figure 3: Orbital energy conservation with atmospheric drag.

As shown in the figures, larger timesteps produce a greater discrepancy between the orbital mechanical energy change and the energy change calculated from atmospheric drag. However, for smaller timesteps such as 15 s and 7.5 s, the difference becomes very small.

These results indicate that the numerical solution approaches the expected physical energy balance as the timestep decreases. Although this comparison does not independently verify the accuracy of our atmospheric density or drag

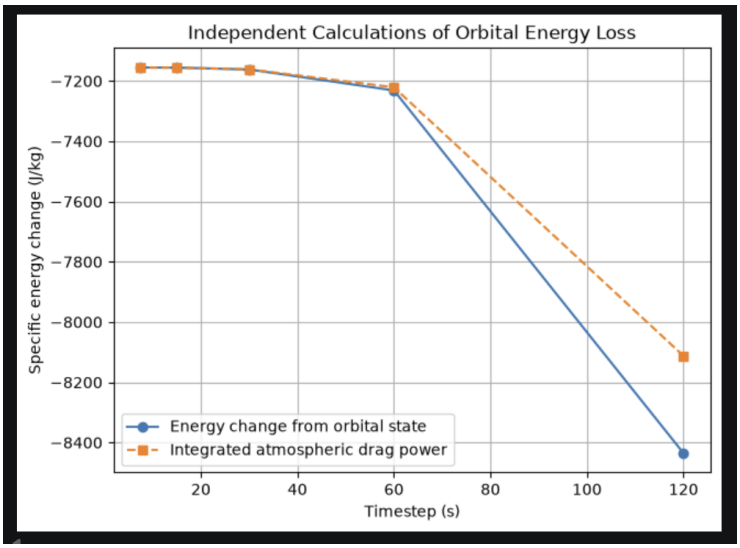


Figure 4: Energy balance convergence with atmospheric drag continued.

coefficient models, it confirms that the numerical integration is consistent with the expected energy loss due to atmospheric drag.

3.2 Finding an Accurate Timestep

Having established that RK4 produces accurate results for sufficiently small timesteps, we next investigate which timestep is appropriate for our orbital decay simulations.

To accomplish this, we simulate satellites starting from circular orbits at altitudes of 200 km and 250 km. For each initial altitude, we calculate the time required for the satellite to decay to an altitude of 180 km using timesteps of 7.5 s, 15 s, 30 s, and 60 s.

Since an exact analytical solution for the orbital decay problem is not available, we use the smallest timestep, 7.5 s, as our numerical reference solution.

We calculate the relative difference between each timestep and the reference solution using

$$\epsilon_t = \frac{|t_{\Delta t} - t_{7.5}|}{t_{7.5}} \times 100\% \tag{78}$$

where $t_{\Delta t}$ is the predicted decay time for a particular timestep and $t_{7.5}$ is the decay time predicted using the 7.5 s reference timestep.

```
========== TEST 4: ORBITAL DECAY ==========
Drag coefficient restored: CD = 2.2

Initial altitude: 200 km
dt (s)  T180 (months)  T120 (months)  V120 (m/s)
  60.0     0.01570662     0.03514124    7817.30188
  30.0     0.01574068     0.03551754    7829.22226
  15.0     0.01574313     0.03553457    7830.72512
   7.5     0.01574327     0.03553563    7831.04003

Initial altitude: 250 km
dt (s)  T180 (months)  T120 (months)  V120 (m/s)
  60.0     0.17025365     0.20034369    7832.04565
  30.0     0.17227315     0.20229670    7829.30022
  15.0     0.17227688     0.20235360    7832.80505
   7.5     0.17227706     0.20235660    7833.12071

Initial altitude: 300 km
dt (s)  T180 (months)  T120 (months)  V120 (m/s)
  60.0     0.73455784     0.77275275    7829.97687
  30.0     0.74295327     0.78302407    7825.36710
  15.0     0.74493652     0.78326941    7836.69858
   7.5     0.74493725     0.78327819    7837.02911
...
  60.0         1.393328         1.343767         0.089986
  30.0         0.266329         0.032443         0.148806
  15.0         0.000099         0.001121         0.004217
   7.5         0.000000         0.000000         0.000000
```

Figure 5: Comparison of time to decay to a 180km orbit from various starting altitude.

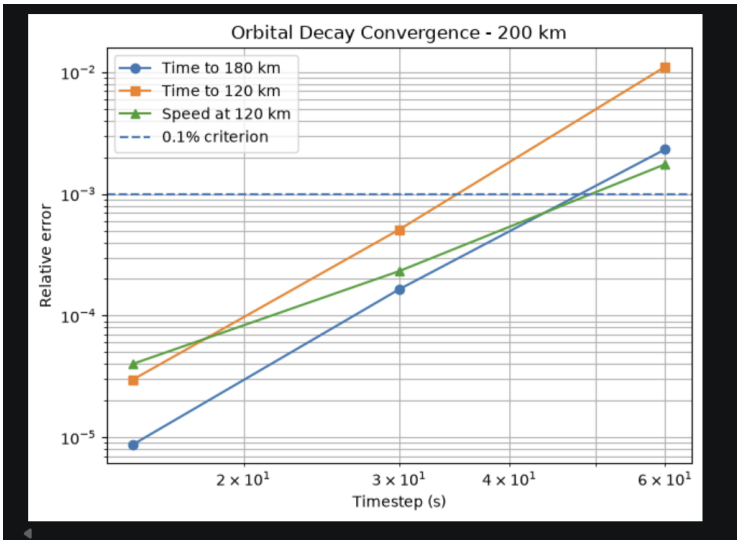


Figure 6: Orbital decay timestep convergence results for r=200km.

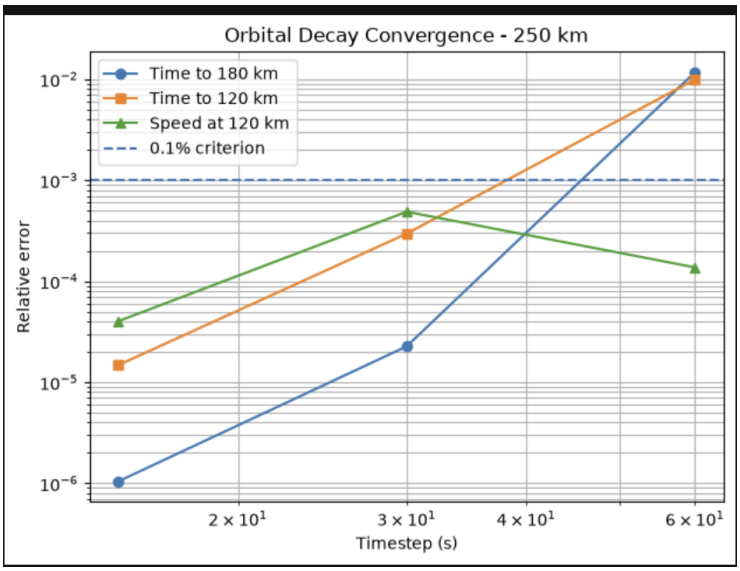


Figure 7: Orbital decay timestep convergence results for r=250km.

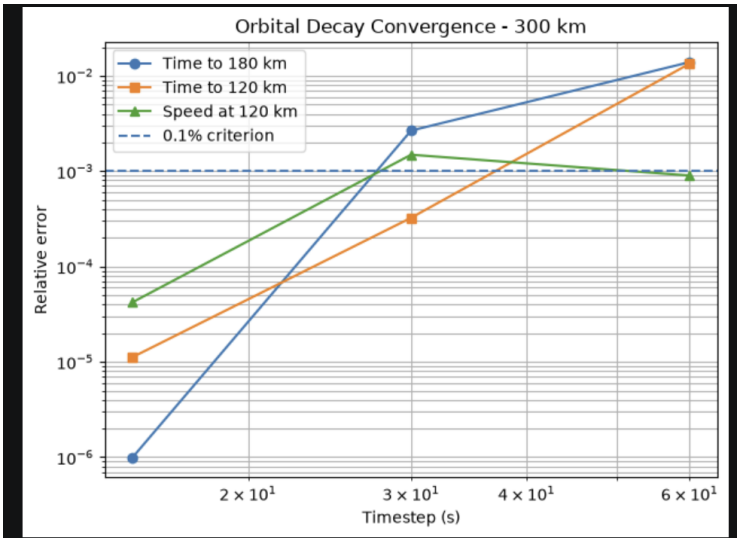


Figure 8: Orbital decay timestep convergence results for r=300km

As shown in the figures, smaller timesteps such as 15 s and 30 s produce results close to the 7.5 s reference solution, while the 60 s timestep shows greater deviations.

This agrees with our previous verification results, where larger timesteps produced significantly greater position errors in the drag-free circular orbit simulation.

Another important observation is that the sensitivity to timestep size can increase for higher initial altitudes. As the initial altitude increases, the satellite generally takes longer to decay, requiring more RK4 integration steps. Consequently, numerical errors may accumulate over a longer simulation period.

Our results indicate that timesteps producing sufficiently small errors for lower-altitude simulations may show greater deviations for higher-altitude cases.

These comparisons allow us to select a timestep that balances computational efficiency with the desired level of numerical accuracy.

4 Results

The RK4 method previously developed and verified was used to answer the original engineering question. Three simulations were performed with a timestep of $\Delta t = 15\,s$, an orbital lifetime analysis based on initial altitude, an orbital lifetime analysis based on solar panel configuration, and a reentry velocity analysis based on initial altitude. These simulations helped answer our subquestions posed at the very beginning, and thus the total engineering question. Orbital lifetime was chosen as the time it takes to reach $120\,km$, a standard height for the "reentry" designation

4.1 Effect of Initial Altitude on Orbital Lifetime

For this simulation, we wanted to investigate the initial altitude's effect on orbital lifetime. This was one of the two variables that was identified in the original engineering premise. Five initial circular orbits were tested: 150, 200, 250, 300, and 350 km. All other parameters were kept constant and the simulations terminated at $120\,km$. Seen in Figure 9, the relationship is nonlinear, with orbital lifetime increasing rapidly with each jump in initial altitude. In the $100\,km$ increase between $250\,km$ to $350\,km$, we see a 12x jump in orbital lifetime

This is likely due to the atmospheric drag that varies with air density. Air density increases significantly as altitude decreases, resulting in a quicker lifetime. High initial altitudes allow the satellite to stay above the $120\,km$ threshold for longer. Since density does not decrease linearly with altitude, this also explains the nonlinear relationship. From this simulation, we can determine that the initial orbital altitude is critical to mission lifetime. Even small changes in altitude can create numerous benefits or disadvantages for the mission lifetime. This demonstrates the importance of choosing proper initial conditions when designing long term LEO missions

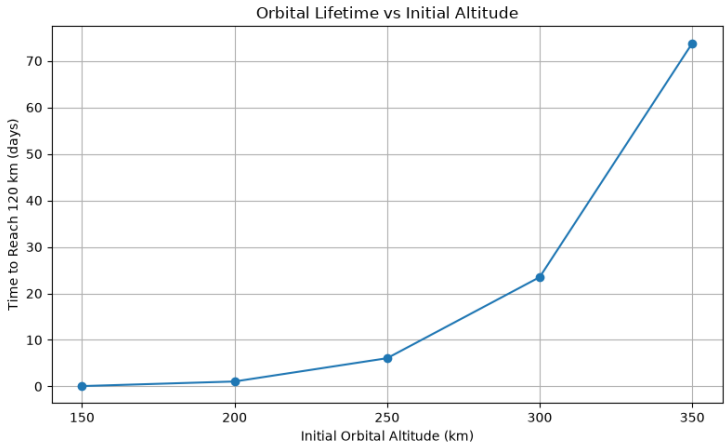


Figure 9: Orbital lifetime results at the 5 initial altitudes

4.2 Effect of Solar Panel Configuration on Orbital Lifetime

For this simulation, we wanted to investigate how the solar panel configuration affects orbital lifetime. This was the second variable identified in our original engineering premise. The initial altitude was held constant at $250\,km$, while four different cross-sectional area configurations were tested: fully retracted, variable area, constant average area, and maximum area. All other parameters were kept constant, and the simulations terminated at $120\,km$. As seen in Figure 10, the relationship shows that increased cross-sectional area generally results in a shorter orbital lifetime. The fully retracted configuration had the longest lifetime at approximately 11.22 days, while the maximum area configuration had the shortest at 2.25 days. Our original variable-area model lasted 6.07 days, approximately 8.1% longer than the constant average area configuration

This is due to the relationship between atmospheric drag and cross-sectional area in our original IVP. Increasing the exposed area of the satellite increases the drag force, resulting in a quicker loss of orbital energy and thus a shorter orbital lifetime. The variable-area configuration changes the exposed area throughout the orbit, allowing the satellite to experience less drag during certain portions of its trajectory. The difference between the variable and constant average area configurations also suggests that where the panels are exposed throughout the orbit has an effect on orbital lifetime. From this simulation, we can determine that solar panel configuration is an important factor when designing long term LEO missions. While solar panels are necessary for power generation, their additional drag can significantly reduce mission lifetime. This demonstrates the importance of considering solar panel deployment when making engineering design decisions

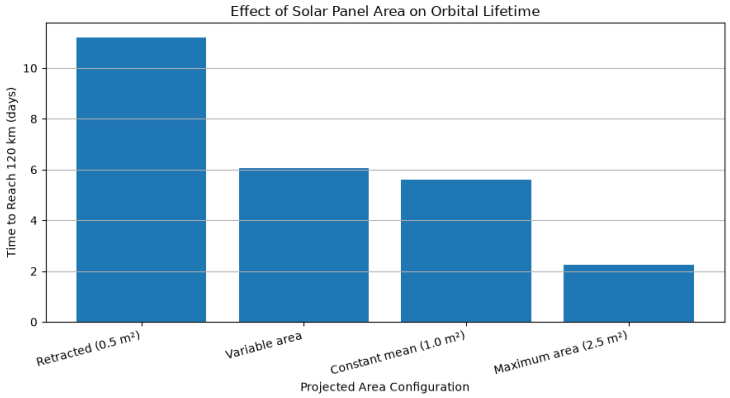


Figure 10: Orbital lifetime results with different solar panel configurations

4.3 Effect of Initial Altitude on Reentry Speed

For the last simulation, we wanted to investigate how initial orbital altitude affects the satellite's velocity when it reaches the 120 km threshold. This was the second output identified in our original premise, alongside orbital lifetime. The same five initial circular orbits from the first simulation were tested, with all other parameters kept constant. The magnitude of the velocity was calculated at the final altitude to determine the satellite's speed. Seen in Figure 11, the relationship shows a slight increase in final speed with increasing initial altitude. At 150 km the final speed was approximately 7821 m/s while at 350 km it increased to approximately 7843 m/s. This is only a 0.29% increase, despite the significant difference in initial altitude

This is likely due to the fact that all simulations terminate at the same altitude of 120 km. Although the satellites start at different altitudes and experience different amounts of orbital decay, they all reach a similar final orbital radius. The effects of gravity at this radius are therefore the same, while differences in the initial orbital energy and energy lost to atmospheric drag result in slightly different final speeds. Comparing this to our initial altitude simulation, we can see that the initial altitude has a significantly greater effect on orbital lifetime than final speed. While increasing the initial altitude from 250 km to 350 km resulted in approximately 12x the orbital lifetime, the final speed changed very little. From this simulation, we can determine that initial orbital altitude is critical to mission lifetime, but has a much smaller effect on the satellite's final speed at the 120 km threshold. This helps further explain how initial conditions affect different aspects of orbital decay in LEO

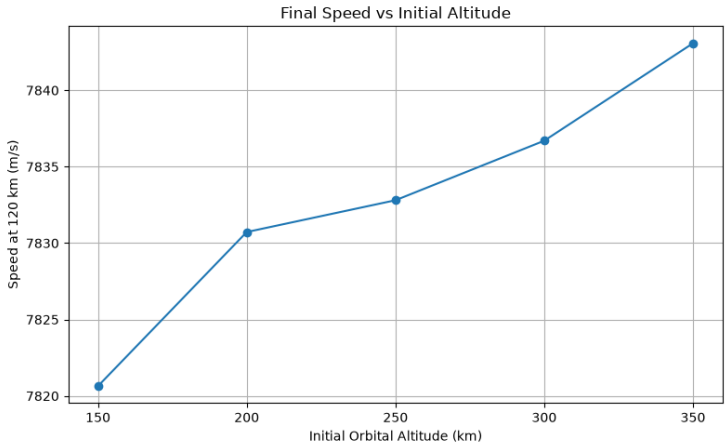


Figure 11: Reentry velocities at the 5 initial altitudes

5 Contributions

The project was split into four parts, one for each requirement. The distribution of contribution for each rubric item is as follows:

- **Item (1) Premise:** Abel – 100%
- **Item (2) Method:** Aathreya – 50%, Adam – 50%
- **Item (3) Implementation:** Burak – 100%
- **Item (4) Results:** Abel – 100%

The total distribution across the entire project is as follows:

- Aathreya – 20%
- Abel – 30%
- Adam – 20%
- Burak – 30%

GitHub Repository:
`https://github.com/abel-yoh/ae370-orbital-decay.git`

6 Acknowledgments

Generative AI was used primarily for the coding portion of this project, both implementation of the method and for generating the graphs for meaningful results. It was also used to gather general information for context regarding the problem (standard values for many variables)